Lab Session 01

Demonstrate DSP toolbox to synthesize and analyze signals

MATLAB Fundamentals

a. MATLAB software will be used to implement and simulate digital signal processing algorithms. The name MATLAB stands for MATrixLABoratory. Originally MATLAB was developed to deal with only one single but universal data type: the matrix. A matrix is defined by

- Its associated name
- The number of rows
- The number of columns
- The value of all matrix elements

The matrix data type includes vectors, for the case when either of the number of rows or the number of column equals to 1; and it includes scalars when both the number of rows and columns equal 1. Furthermore, matrices can be either real-valued or complex valued, which is the most general case. This Lab will cover MATLAB fundamentals. It is recommended to try out introduced methods while solving the described tasks.

b. This Lab also covers the difference between a script file and a function file by plotting several signals. So, basically we would learn two things. First, plotting of signals and second, difference between script and function file. The command window environment now shift to the script or function file environment. The main objective of making script or function file is to store data for future usage as command window do not have the facility of data storage after exit. As the MATLAB starts the command window initializes and as soon as the MATLAB terminates i.e. closed, the data on command window screen is cleared. While a script file or function file can be stored for future reference.

How to get help?

There are several methods in MATLAB to access texts, which explain the usage and behavior of given functions:

- The **HELP** command is the basic help feature to get help of a function. For example, to get help on the SUM function, type at the command prompt:

help sum

- **HELPWIN** is used in the same manner as help, but it opens a separate window with advance search facilities and links to related topics.
- **HELPDESK** uses a window of an installed web browser to display the help texts.
- If the name of a command is not exactly known, the **LOOKFOR** function helps by seeking through the first comment line of all available functions and listing all functions where a desired expression is found.

Statement, expressions and variables:

MATLAB is an expression language. The expressions you type are interpreted and evaluated. MATLAB statements are usually of the form
variable = expression, or simply expression
Expressions are usually composed from operators, functions and variable names. Evaluation of

the expression produces a matrix, which is then displayed on the screen and assigned to the variable for future use.
A statement is normally terminated with the carriage return. However, a statement can be continued to the next line with three or more periods followed by a carriage return. On the other hand, several statements can be placed on a single line if separated by **commas** or **semicolon.**

- **The variable ANS:**

If the variable name and = sign are omitted, a variable **ANS** (for answer) is automatically created to which the result is assigned.

- **Suppressing Screen Output:**

If the last character of a statement is a semicolon, the printing is suppressed, but the assignment is carried out. This is essential in suppressing unwanted printing of intermediate results.

- **Colon Notation:**

Colon notation is used to generate vectors, and reference submatrices, known as subscripting. Creative use of this feature makes MATLAB programming code simple, readable and minimizes the use of loop, which slows MATLAB.

- An index vector is created by using the colon notation

first : last
For example, [1:5] creates the vector [12345]

- An spacing can be introduced between two elements of a vector by using the colon notation

first : spacing : last
For example, [1: 2 : 5] creates the vector [1 3 5].

Workspace:

The contents of all the variables are stored in the MATLAB workspace. This is the memory region allocated for variable.

- The command **WHO** lists all variable which currently exist in the workspace.
- **WHOS** additionally lists the size and amount of allocated memory.
- The entire workspace or single variable can be cleared by using the **CLEAR** command.

Elementary operations:

Algebraic expressions can be formed by the following basic operations:

- Transpose of a vector/matrix can be produced by .*

e.g. B.*

- + is used to add two vectors/matrices of identical size, or a vector/matrix and a scalar, e.g.

[3 2; 4 5] + [7 4; 9 6]

- - subtracts two vectors/matrices of identical size, or a vector/matrix and a scalar, e.g.

2-B

- .* performs element-wise multiplication of two vectors/matrices of identical size, or a vector/matrix and a scalar. For example, to square all elements of B, we may write

B.*B

- ./ performs element-wise division of two vectors/matrices of identical size, or a vector/matrix and a scalar. For example, the reciprocal of all elements in B is computed through

1./B

- * performs vector/matrix multiplication. The number of columns in the first vector/matrix must equal to the number of rows in the second. Example:
  B * B

Real and complex matrices:

In general, any matrix within the MATLAB can be complex-valued. However, for efficient storage, MATLAB distinguishes between real-valued and complex-valued matrices. Real valued matrices are matrices where the imaginary parts of all matrix elements are zero. The following essentials must be known to deal with complex-valued matrices:

- The variables i and j are assigned, by default, the value $i = \sqrt{-1}$. This is used to define the complex values. For example,
  5 + j * 10
  Generates a complex-valued variable.
- The real part of a complex-valued matrix can be extracted by using the function REAL and imaginary part can be extracted by using the function IMAG. Both functions deliver the real-valued matrices as outputs.
- The function CONJ is used to produce the complex conjugate of a matrix.
- The special character **'** generates the complex conjugate transpose of a matrix, the hermitian matrix. This character is written after an expression or variable.
  For example, the hermitian of a matrix A can be obtained through: A'

M-files:

MATLAB can execute a sequence of statements stored in a file. Such files are called **M-files** because they have "**.m"** extension as the last part of their file name. There are two types of M-types:

- Script files
- Function files

Script file:

In a script file, script is a sequence of commands as they could be entered at the prompt. The script allows to execute the entire sequence multiple times in a comfortable way, or to test modified versions.

Function file:

In a function file, function has the additional feature of passing parameters. On calling a function, it may read input arguments and after execution it may return output values. The **"FUNCTION"** command specifies the input and output parameters of a function. It must be very first command in a function file. Only comment may be written before the **FUNCTION** command. Any text after a **%** sign inside an m-file is comment.

Flow control:

In MATLAB flow control means that a program may not only be executed straight down from top to bottom but with branches and repetitions.

- The **FOR** function allows to create a loop which is controlled by a counter.

- More general exit conditions can be implemented in a **WHILE** loop.
- The **IF** command allows for conditional programming, together with **ELSE.**
- All these commands come along with an **END** command which acts as a closing brace, together with the opening brace, i.e. the introducing **FOR, WHILE** or **IF** command.

Plotting of signals:

A plot is any graphic display you can create within a figure window. A figure is a MATLAB window that contains graphic displays (usually data plots). You create figures explicitly with the figure function, and implicitly whenever you plot graphics and no figure is active. By default, figure windows are resizable and include pull-down menus and toolbars. Plots can display tabular data, geometric objects, surface and image objects, and annotations such as titles, legends, and color bars. Figures can contain any number of plots. Each plot is created within a 2-D or a 3-D data space called axes. You can explicitly create axes with the axes or subplot functions.

A graph is a plot of data within a 2-D or 3-D axes. Most plots made with MATLAB functions and GUIs are therefore graphs. In MATLAB we can visualize the results on graphs by using several commands. Plot command is generally used to generate 2-D graph of continuous time signal while stem command is used to generate 2-D graph of discrete time signal. Subplot command can be used to divide the graphical screen in to number of columns and number of rows. The plot, stem, subplot commands are very helpful in analysis which we would discover in this lab as well.

Plot command:

MATLAB plotting functions and tools direct their output to a figure window. Each figure is a separate window that you can dock in the desktop, and collect together with other plots in a Figure Group. To illustrate the basic components of a graph, execute the following code to create a plot of a family of sine curves:

```
x = 0:.2:20;
y = sin(x)./sqrt(x+1); y(2,:)
= sin(x/2)./sqrt(x+1); y(3,:)
= sin(x/3)./sqrt(x+1);
plot(x,y)
```

The resulting figure contains a 2-D set of axes. This graphic identifies the components and tools of a figure window.

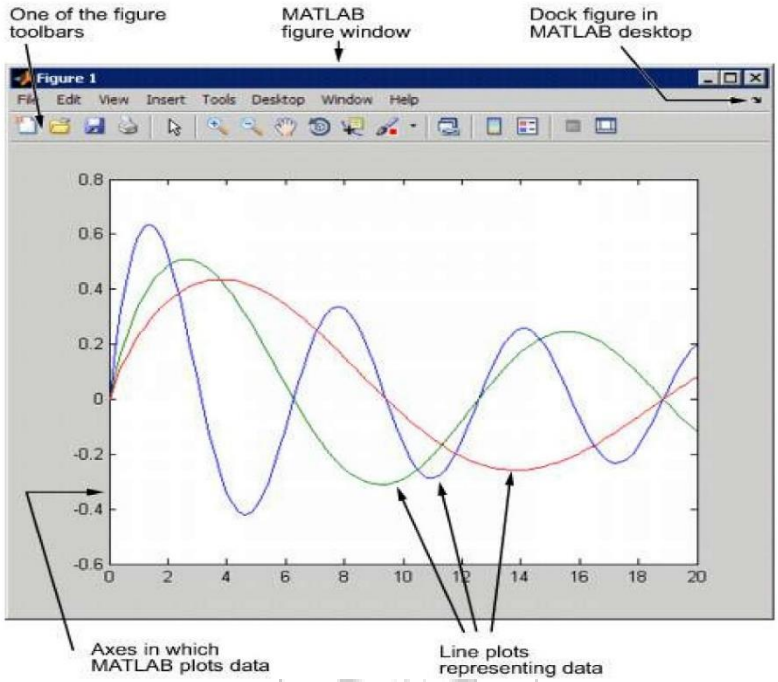


Fig. 1.1: Generation of Continuous time signal using Plot command

The plot function uses a default line style and color to distinguish the data sets plotted in the graph. You can change the appearance of these graphic components or add annotations to the graph to present your data in a particular way.
This command is used to generate two-dimensional graph on screen. The syntax used for plot command is

Plot(x,y)

Where x defines the x-axis and y defines the y-axis. Labeling on a graph is possible with the help of following commands

- xlabel: It defines the label on x-axis
- ylabel: It defines the label on y-axis
- title: It is used to give a title to a graph

The plot command can generate more than one signal on a single graph with different color. For detailed information of plot command, use help on MATLAB.

Stem command:

The stem command shares all the features of a plot command, but the only difference is the type of signal which is generated on the graph. The stem command is used for the discrete time signals while the plot command is used for the continuous time signals. Following figure is an example of stem command:

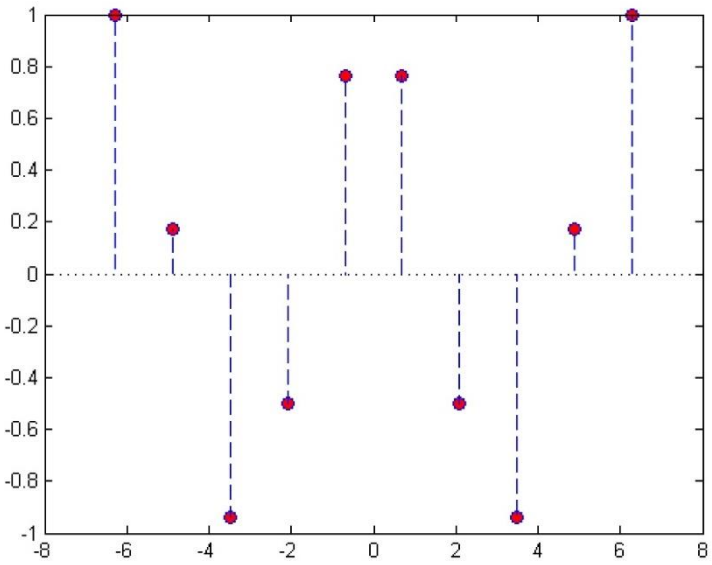


Fig. 1.2: Generation of Discrete time signal using Stem command

Subplot:

A graphical screen can be divided in to multiple sub-screens by subplot command. The screen is divided in to m by n matrix by using following syntax

Subplot(m,n,p)

Where m and n are the dimensions in which the figure screen would be divided and p defines the position.

Following figure illustrate the example of subplot command.

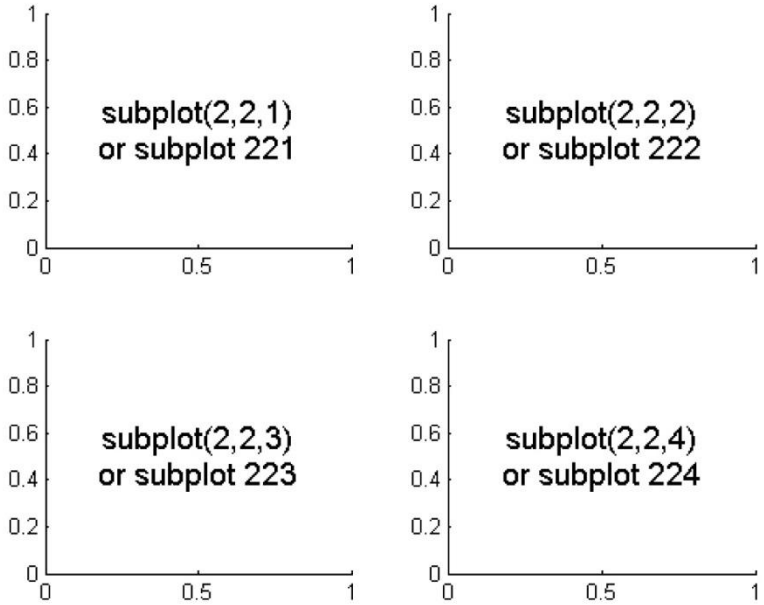


Fig. 1.3: Generation of 4 plots on a single figure using subplot command

Harmonics:

Harmonics are integral multiples of a signal. Consider the signal cos ($w_o t$). Its harmonics are cos ($2\ w_o t$), cos ($3\ w_o t$) is the fundamental harmonic

Even & odd harmonics:

If the harmonic of a signal function has an even co – efficient it is called an even harmonic. Otherwise if the co – efficient is odd it is called an odd harmonic.

For $X(t) = \cos(w_o t)$ → $\cos(2 w_o t)$, $\cos(4 w_o t)$, $\cos(6 w_o t)$ ……Even Harmonics
→ $\cos(3 w_o t)$, $\cos(5 w_o t)$, $\cos(7 w_o t)$ ……Odd Harmonics

Even, odd harmonic play an important role in signal construction. Different waveforms can be obtained by adding one type of harmonic e.g. adding infinite odd harmonics of sine/cosine where generates a square wave. Both will have a 90 degree phase shift from each other.

Exercises:

Task # 1: Generate a script file which produces a sine wave with 2 Hz frequency.

Task # 2: Generate a script file which produces a sine wave by taking frequency as an input at run time.

Task # 3: Generate a script file which produces a sine wave with 2Hz, 5Hz, 10Hz and 20Hz frequencies on a single figure by using subplot command. Label the subplots with the relevant frequency to distinguish the signals clearly.

Task # 4: Implement a function as defined below, which produces a sine wave of 1001 samples with spacing of 1 ms.

[x]=sinewave(t)

Task # 5: Generate a function which produces a sine wave by accepting frequency and amplitude as input arguments and return a plot of sine wave for the inputted parameters.

Task # 6: Repeat all above tasks by using stem command rather than plot command.

Task # 7: Write MATLAB function for plotting the 5 harmonics of a signal x(t)=cos(wt) with a fundamental frequency of f = 0.5 Hz. Take t = 0:0.01:2

What are the frequencies and Time periods at the all 5 harmonics?

Task # 8: Try all help methods to get help of any function.

Task # 9:
- **(a)** Assign 10 samples, from 0 to 9, of time to the vector **t**.
- **(b)** Assign a vector of samples without assigning it to a variable.
- **(c)** Assign 10 samples, from 0 to 9, of time to any vector without printing it to screen.

Task # 10: Investigate the difference between multiplication***** and element-wise multiplication**.*** of vectors/matrices.

Task # 11: Generate a complex-valued matrix

a = ones (1,10)+ i* (1 : 10)

and calculate the absolute square of all elements of this matrix.

Task # 12: Implement a function

[x]=sinewave(t)

Which produces a sine wave of 1001 samples with spacing of Δt= 1ms.

Task # 13: Use MATLAB **help** to get familiar with the syntax of **FOR, WHILE** and **IF-ELSE** statement.

Task # 14: Use **FOR** loop to generate a single vector **y**, which is a digitized unit sine wave with ten samples per cycles, with 100 elements.

NED UNIVERSITY OF ENGINEERING & TECHNOLOGY
KARACHI

NED UNIVERSITY OF ENGINEERING & TECHNOLOGY
KARACHI